

Host-based Mitigation of Keystroke Injection Attacks via Emulated Peripherals

Minjin Kim
Computer Science
Virginia Tech
Alexandria, VA, United States

Darian Morack
Computer Science
Virginia Tech
Alexandria, VA, United States

ABSTRACT

Operating systems tend to trust any connected device that portrays itself as an HID. Because of this security flaw, malicious USB devices can be used to quickly and discretely compromise a system. Multiple defenses have already been designed before to combat this, but they are all lacking in some way. Accordingly, we developed a software-based defense which utilizes keystroke timing analysis in combination with a user-configurable hotkey and string blacklist. Our defense shows promise based on initial testing via our custom dataset, producing a large amount of detections. These results suggest a composite approach like ours may be highly effective at defending against these types of attacks, though more development and testing is necessary.

KEYWORDS

BadUSB, Rubber Ducky, HID Emulation, Keystroke Detection

1 Problem Statement

Operating systems have a fundamental security issue where they just trust any connected device claiming to be a standard keyboard. Therefore, unverified USB HIDs (Human Interface Device) like compromised wireless receivers or BadUSBs (malicious physical USB devices that emulate HIDs), can be used to inject malicious, high-speed keystrokes to execute arbitrary commands.

Many of the existing defense mechanisms mostly rely on USB port blocking, DLP which is Data Loss Prevention, or antivirus software that only detects payloads after execution begins, failing to stop or detect the initial input injection itself. Those current defenses are few in number and sometimes lack information such as source code and actual effectiveness, so further development and testing is needed to determine their efficacy. Our goal was to develop a new algorithm/software to be installed on host systems to detect those inputs, and do so in a way that will support further

development and improvements, rather than stagnate after initial development.

2 Related Work

A 2024 paper titled USB Rubber Ducky Hunter A Proactive Defense Against Malicious USB Attacks Domain: Cybersecurity [1] is closely related to our work. USB Rubber Ducky¹ Hunter captures keystroke inputs and dynamically analyzes keystroke input speed to detect whether an HID is malicious or not. They implemented 4 different modes for the program: Normal (displays detection messages), Paranoid (blocks further keyboard input upon detection), Sneaky (Removes some keyboard inputs after detection), and Log-Only.

As their approach only uses keystroke timings, it is vulnerable to attackers adjusting their keystroke timings to circumvent the defense. Our approach adds on a blacklist to serve as another obstacle to, which serves to complicate the process of adapting an attack to bypass our defense. One benefit to their approach vs. ours is that they do have a mode (paranoid) that does block keyboard input instead of just detecting it.

Other defense mechanisms are mentioned in the paper as well. They include USBBlock (OS-Level, USB-Packets), Keyblock (Software-based, keystrokes), GoodUSB (user manually approves device), USBWall (hardware-based, middleman between HIDs and system), and USB Check-In (CAPTCHA-like approval before use is allowed).

¹ A Rubber Ducky is a usb device that disguises itself as a keyboard and injects keystrokes using DuckyScript.

3 Threat Model

Our designated threat model entails the attacker having physical access to the system in a way that they can insert a malicious usb device into the system. The malicious usb

device contains a program that can bypass endpoint security by camouflaging as a keyboard and inject commands that are already programmed to compromise the system. The attacker cannot otherwise use/access the system itself, though further improvements to the software could expand the threat model to include that situation.

Our designated threat model entails the attacker having physical access to the system in a way that they can insert a malicious usb device into the system. The malicious usb device contains a program that abuses the operating system's trust in keyboard input by camouflaging as a standard keyboard and injecting pre-programmed commands. The attacker cannot otherwise use or access the system directly, and does not have remote access or administrative credentials. The target system is assumed to be powered on and able to receive keyboard input. Further improvements to the software could expand the threat model to include stronger response actions, such as blocking input after detection.

Our designated threat model entails the attacker having physical access to the system in a way that they can insert a malicious USB device into the system. The malicious USB device contains a program that camouflages itself as a standard keyboard and injects pre-programmed keystrokes into the target machine. These keystrokes may include opening a command prompt, typing commands, using hotkeys, or navigating to a URL.

The attacker cannot otherwise use or access the system directly, and does not have remote access or administrative credentials. The target system is assumed to be powered on and able to receive keyboard input. Our current defense focuses on detecting this injected keyboard behavior through timing-based detection and blacklist-based detection. It does not prevent the USB device from being connected, and it does not block input after detection in the current implementation. Further improvements to the software could expand the threat model to include stronger response actions, such as blocking input after detection.

4 Approach

Our approach is to combine timing-based detection with behavior-based blacklist detection. Keystroke injection attacks often look different from normal typing because they can be faster, more consistent, and more command-focused. If the system only relies on speed, an attacker could slow down the payload and avoid detection. Because of this, our system also checks for suspicious strings, hotkeys, and hotkey-string pairs.

The main idea is to detect both timing patterns and behavior patterns. For example, GUI+R by itself can be normal, and cmd by itself can also appear in a normal context. However, GUI+R followed by cmd is more suspicious because it looks like the device is opening the Run dialog and trying to execute a command. This gives the daemon more context than only checking typing speed or only checking keywords.

A key part of our design is that the blacklist is adjustable. Strings, hotkeys, and pairs are stored separately from the daemon logic, so detection rules can be added or changed without rewriting the main code. The current version focuses on detection and logging only. It does not block input yet because we wanted to first evaluate whether the detection logic works before adding blocking behavior.

5 Implementation

The implementation has three main components: the HID runner, the dataset, and the daemon. The HID runner generates controlled keystroke input, the dataset defines what input should be sent, and the daemon monitors the host machine and records detections. The project code, dataset, and implementation files are available in our project repository [7].

5.1 Human Interface Device (BadUSB)

For the BadUSB side, we used a Raspberry Pi Zero 2 W as an emulated USB keyboard. The Raspberry Pi was set up with CircuitPython so that it could act as a Human Interface Device instead of a normal storage or computing device [2] [3]. After setting up CircuitPython, we used the `adafruit_hid` library to send keyboard input from the device to the target computer.

The HID runner is written in circuitPython. Its main job is to read the testing dataset and send the keystroke events from that file. It supports four main event types: TEXT, KEY, COMBO, and DELAY. TEXT is used to type strings, KEY is used for single keys such as Enter or Space, COMBO is used for key combinations such as GUI+R or CTRL+L, and DELAY is used to wait between actions.

We also added typing speed control to the HID runner. This allows the same input sequence to be tested at different speeds instead of only running on fixed payload. The runner can use fixed speed profiles, and it can also run a human-like random typing mode with random delays and pauses. This was useful because keystroke injection attacks are usually faster and more consistent than normal typing, but an attacker could also slow down the payload to make it look more realistic.

During development, we wrote and edited the code from a laptop, copied the CircuitPython files onto the Raspberry Pi, and then plugged the device into the target machine as a keyboard. Once connected, the device could automatically run the script and send the test input.

5.2 Dataset

We created dataset.json ourselves because we could not find a public dataset that fit our testing needs. The dataset we were going to use, which was hak5 payload datasets [5], focused on payload scripts, but we needed structured test sequences for evaluating our daemon. Our dataset includes normal text, normal shortcuts, suspicious command-like strings, hotkey-string pairs, false positive cases, and evasive cases.

The dataset is stored in dataset.json [4]. It is the input dataset used by the HID runner to decide what keystrokes should be sent. We used the dataset to make testing more repeatable because the same sequences can be run multiple times at different speeds.

The dataset contains several types of test sequences. It includes normal text, normal shortcuts, suspicious command-like strings, hotkey and string combinations, false positive tests, and borderline or evasive tests. Normal text and normal shortcuts are included to check whether the daemon incorrectly flags regular user behavior. Suspicious command-like strings include examples such as cmd, powershell, whoami, ipconfig, sudo, curl, and wget. These strings are not executed as real attacks in the dataset, but they are useful for testing whether the daemon can recognize suspicious command-focused input.

The dataset also includes hotkey and string combinations. This is important because many BadUSB-style attacks are not just one suspicious word. They usually follow a sequence, such as opening the Windows Run dialog with GUI+R and then typing cmd or powershell. Because of this, the dataset includes sequences that test those behavior patterns.

We also included false positive and evasive cases. The false positive cases use normal sentences that contain suspicious-looking words, such as admin, server, download, or terminal. This helps test whether the blacklist is too broad. The evasive cases include modified strings such as p0wershell, power shell, c m d, and who am i. These cases are useful because they show the limits of simple substring matching and help identify what needs to be improved later.

5.3 Daemon

The daemon is the host-side detection component. It is written in Python and uses pynput to monitor keyboard input globally [6]. The daemon currently focuses on detection and logging only. It does not block keystrokes yet because we wanted to first test whether the detection logic works before adding blocking behavior.

The daemon uses both timing-based detection and blacklist-based detection. For timing-based detection, it tracks completed words and calculates the average character delay over a recent word window. If the average delay is below the configured threshold, the daemon records a speed detection. It also checks timing consistency because automated HID input can be suspicious not only because it is fast, but also because the timing can be too consistent compared to normal human typing.

For blacklist-based detection, the daemon checks blocked strings, blocked hotkeys, and blocked hotkey-string pairs stored in the SQLite database. A blocked string could be something like cmd or powershell. A blocked hotkey could be something like GUI+R. A blocked pair could be something like GUI+R followed by cmd. The pair logic gives the daemon more context because a single hotkey or a single word may be normal by itself, but the combination can be more suspicious.

When the daemon detects suspicious input, it writes the result into the SQLite logs. The project also separates the daemon logic from the database interaction layer, so blacklist entries and logs can be managed without putting all of the SQL logic directly inside the daemon. Overall, the daemon works as a background monitor that watches keyboard activity, checks timing behavior and blacklist rules, and records detections for later evaluation.

5.4 Database

We designed a database² to store both the blacklist entries and the logs produced by the daemon. The blacklist contains entries for blacklisted strings/words, hotkeys, and hotkey + string combinations. It also allows the user to block or unblock each of these entries individually while keeping them within the database. The database is already pre-populated with a large number (>100) of common hotkeys so the user does not need to enter them themselves.

We also developed a repository to serve as a sort of interface with the database to support future additions or adjustments that wouldn't require refactoring large portions of the codebase.

² A relational schema for the database can be found in Appendix A.

5.5 User Interface

We developed a separate UI² using Streamlit to allow easy adjustments to the blacklist and allow the user to view and delete log entries. For strings and hotkeys, the user can update or delete entries within the database or add a new entry. For hotkey-string combinations, the user can update or delete entries already in the database or choose from already present hotkeys and strings to add a new pair to the database. Updating any of those entries describes only adjusting the blocked state of the selected entry.

Both types of log entries (blacklist detections and keystroke speed detections) are also viewable. The user can further delete entries in an easy fashion as well to clear the database.

Further improvements to the UI could be made such as adding in-row delete buttons + blocked status checkboxes rather than a selection dropdown and a clear all button for the logs.

³ Screenshots of the UI can be found in Appendix B.

6 Work Performed

After implementing the HID runner, dataset, daemon, database, and UI, we performed an initial end-to-end validation test using the 240 ms fixed-delay profile. The HID runner supports multiple speed profiles, including 340, 300, 280, 260, 240, 220, 200, and 180 ms, as well as a human-like random typing mode. However, for this report, we focused on the 240 ms profile first to verify that the full pipeline worked correctly before treating the system as a full benchmark.

The goal of this test was to check whether the Raspberry Pi could inject the JSON dataset, the daemon could monitor the input, the timing and blacklist rules could generate detections, and the SQLite database could store the results. We compared the behavior intentionally included in the dataset with the detection logs recorded by the daemon. Since the dataset included normal text, normal shortcuts, suspicious command strings, hotkey-string pairs, false positive cases, and evasive strings, the test was not limited to only obvious malicious commands.

Table 1 summarized the detection logs from the 240 ms validation run. The daemon recorded both timing-based and

blacklist-based detection logs. For timing-based detection, the daemon recorded 5,614 total speed detection log entries. Out of these, 816 were direct speed-threshold detections, and 4,798 were timing-consistency detections. The direct speed threshold was 240ms, and the consistency rule used a four-word window with a standard deviation threshold of 8ms.

Table 1. Detection logs from the 240 ms validation

Detection Type	Count
Total speed detection logs	5,614
Direct speed-threshold detections	816
Timing-consistency detections	4,798
Total blacklist detection logs	3,569
String blacklist detections	3,470
Hotkey detections	22
Hotkey-string pair detections	77

For blacklist-based detection, the daemon recorded 3,569 total blacklist detection logs. Most of these were string blacklist detections, but the daemon also recorded hotkey detections and hotkey-string pair detections. Common string matches included powershell, https://example.com, power shell, cmd, i p config, p0wershell, who am i, and c m d. The pair detections included GUI+R followed by powershell and CTRL+L followed by https://example.com.

We also made a simple comparison between the unique blacklist patterns in the dataset and the unique patterns that appeared in the detection logs. Out of 28 unique blacklist patterns present in the dataset, 14 appeared in the logs. This shows that the blacklist system detected several expected patterns, but it did not provide complete coverage in this initial validation run. These results matched the types of behavior that were intentionally included in the dataset.

7 Findings and Implications

The 240 ms validation test showed that speed-only detection is limited. As shown in Table 1, timing-consistency detections were much higher than direct speed-threshold detections. This suggests that automated HID input should not only be judged by typing speed, but also by how regular the timing pattern is. An attacker could slow down a payload to avoid a simple speed threshold, but the input may still

look machine-generated if the delay pattern is too consistent.

The blacklist results also showed that behavior context matters. A single hotkey or keyword can be normal by itself, but a sequence such as GUI+R followed by powershell is more suspicious because it shows both the action and the intended command.

From a security perspective, the system adds detection at the input level instead of only relying on tools that detect activity after command execution. This is useful for keystroke injection attacks using emulated USB keyboards because the malicious device abuses the trust given to normal keyboard input. However, the current implementation is still detection-only, so it does not stop the attack after detection.

One limitation of this evaluation is that the database mainly stores detection events, not every input sequence that was executed. Because of this, the results should not be treated as a complete detection accuracy or false positive rate. The blacklist comparison showed partial coverage, which means the rule set and logging format need more refinement in future testing.

8 Conclusion

This project developed a host-based prototype for detecting keystroke injection attacks from emulated USB HID devices. The system combines timing-based detection, blacklist-based detection, a structured JSON dataset, SQLite logging, and a Streamlit UI. The 240 ms validation test showed that the prototype worked from HID input injection to detection logging.

The findings suggest that combining timing analysis with blacklist and hotkey-string rules can provide an additional input-level defense. The current version is still detection-only, so future work should test all speed profiles, tune blacklist and timing rules to reduce false positives, improve the UI, and add a blocking feature after detection.

REFERENCES

- [1] N. T. Arun Jothi, S. Anu, K. Harsha and R. Devi Priya, "USB Rubber Ducky Hunter A Proactive Defense Against Malicious USB Attacks Domain: Cybersecurity," *2024 International Conference on Intelligent Systems for Cybersecurity (ISCS)*, Gurugram, India, 2024, pp. 1-6, doi: 10.1109/ISCS61804.2024.10581045.
- [2] Adafruit Industries. 2026. CircuitPython Libraries. Retrieved May 6, 2026 from <https://circuitpython.org/libraries>
- [3] wirebits. 2024. RPI0-Series-USB-Rubber-Ducky: Setup for Raspberry Pi Zero Series boards to make a USB Rubber Ducky. GitHub. Retrieved Mar 1, 2026 from <https://github.com/wirebits/RPI0-Series-USB-Rubber-Ducky>
- [4] Minjin Kim and Darian Morack. 2026. dataset.json. Host-based-Mitigation-of-Keystroke-Injection-Attacks-via-Emulated-Peripherals. GitHub. Retrieved May 1, 2026 from <https://github.com/minjintiger/Host-based-Mitigation-of-Keystroke-Injection-Attacks-via-Emulated-Peripherals/blob/main/hid/dataset.json>
- [5] Hak5. 2026. USB Rubber Ducky Payloads. GitHub. Retrieved May 6, 2026 from <https://github.com/hak5/usbrubberducky-payloads/tree/master>
- [6] Moses Palmér. 2025. pynput 1.8.1. PyPI. Retrieved May 6, 2026 from <https://pypi.org/project/pynput/>
- [7] Minjin Kim and Darian Morack. 2026. Host-based Mitigation of Keystroke Injection Attacks via Emulated Peripherals. GitHub. Retrieved May 6, 2026 from <https://github.com/minjintiger/Host-based-Mitigation-of-Keystroke-Injection-Attacks-via-Emulated-Peripherals>

keywords: {Training;Detectors;Universal Serial Bus;Rendering (computer graphics);Rubber;Computer security;Monitoring;Bad usb attacks;rubber ducky;keystroke speed monitoring;mode-specific analysis;cybersecurity measures...},

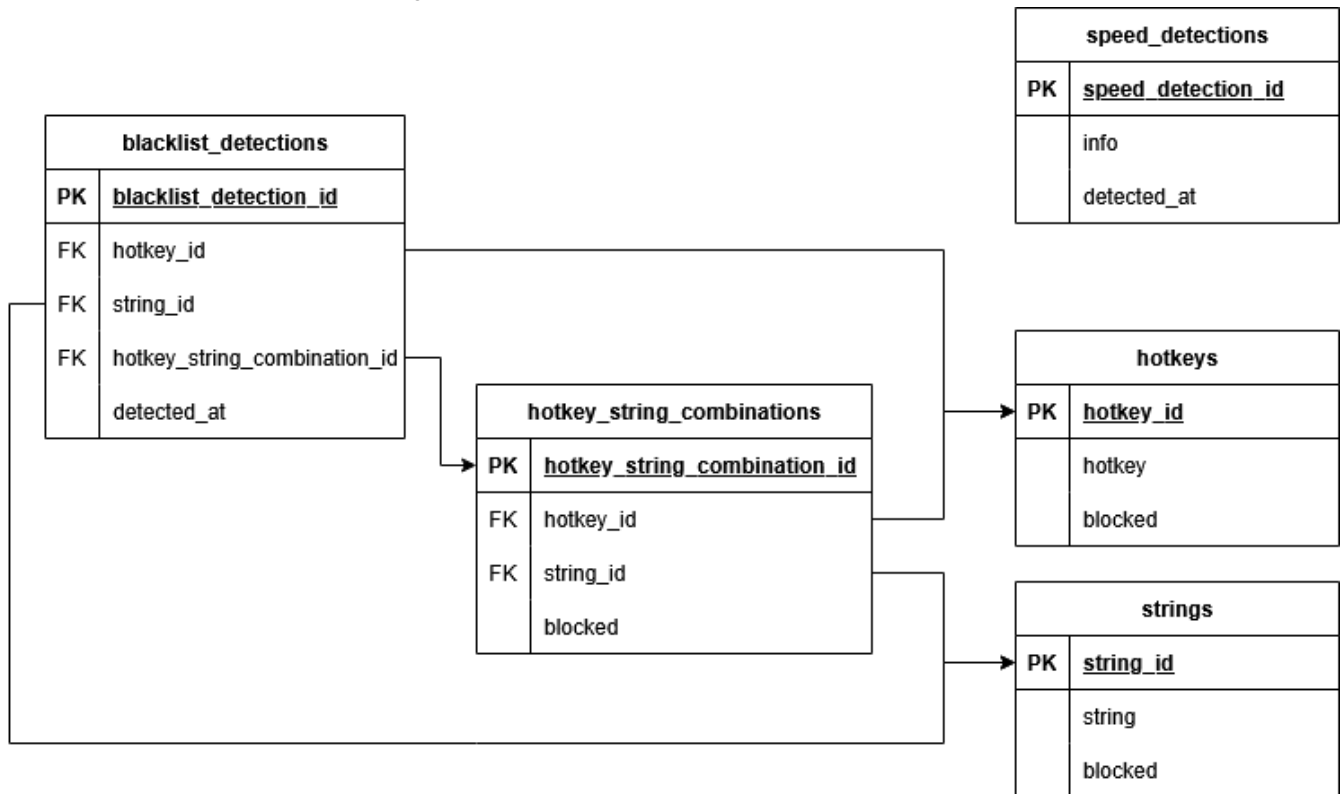
We wrote this report ourselves, not using generative AI technologies.

Repository Availability

Due to security concerns, the project repository is currently private. Access can be provided to the grader upon request if they would like to review the implementation files.

APPENDIX A

A relational Schema of the blacklist + log database:



APPENDIX B

The following are screenshots of the UI for viewing and updating the database:

Hotkeys (same format for strings):

Select a table to view:

Hotkey Blacklist

hotkey	blocked
GUI+R	Yes
GUI+X	Yes
ALT+F4	Yes
CTRL+ESCAPE+SHIFT	Yes
ALT+CTRL+DELETE	Yes
CTRL+L	Yes

▼ Add New Hotkey:

Hotkey

☐ Blacklist

+ Add Hotkey

▼ Update or Delete Hotkey:

Select a hotkey to update:

☒ Blacklist

⊘ GUI+R

✎ Update Hotkey

🗑 Delete Hotkey

Hotkey-String Pairs:

Select a table to view:

Hotkey String Pair Blacklist



hotkey	string	blocked
GUI+R	cmd	Yes
GUI+R	powershell	Yes
GUI+R	taskmgr	Yes
GUI+R	regedit	Yes
GUI+R	ipconfig	Yes
GUI+R	whoami	Yes
CTRL+L	https://example.com	Yes

▼ Add New Hotkey-String Pair:

Select a hotkey:

GUI+R



Select a string:

powershell

☐ Blacklist**+ Add Pair****▼ Update or Delete Pair:**

Select a pair to update:

GUI+R + cmd

☒ Blacklist **Update Pair** **Delete Pair**

Keystroke Speed Detection Log (same format for Blacklist Detection Log):

Select a table to view:

Keystroke Speed Detection Log			
ID	Info	Detected At	Delete
1	Example info!	2024-08-19 06:31:32.779000	
2	A second example of info.	2023-01-21 11:10:19.187000	
3	Yet another info example	2023-08-08 16:26:41.575000	
4	The accompanying dates are random...	2021-08-21 02:32:31.378000	
5	There will be some info like keystroke speed and other stuff here.	2025-12-12 07:41:08.012000	